

Statistics Everywhere

[Code ▼](#)

Foundational Concepts

Often, statistical reasoning can contribute to decisive arguments in matters that seem very difficult to resolve, and even issues that don't appear to have any statistical component to them at all.

[Hide](#)

```
if (!caracas::has_sympy()) {  
  caracas::install_sympy()  
}  
library(caracas)  
library(sets)  
library(tibble)  
library(ggplot2)  
library(dplyr)  
library(quanteda)  
library(DiscreteDatasets)  
library(rstanarm);
```

[Hide](#)

```
def_sym("X", "Y")  
  
cmStats <- modules::use("CmStats.R")$cmStats  
  
set.seed(3456)
```

Statistics is a subject that permeates essentially every area of our lives and world. It is a powerful tool for seeing our world in a more detailed fashion and for making informed decisions, although its subtleties and potential misuses caution us to avoid thoughtless acceptance of statistical conclusions. The recent and expected future development of computer speed and capacity allow us to imagine using statistics with ever more scope and effect. How much information and understanding can we hope to gather from statistical data? How much more meaning would better statistical techniques allow us to find? Statistics is a tool with wide applicability. It has limits that need to be acknowledged and respected, but its potential for helping us find meaning in our data-driven world is enormous and growing.

Frequentist Stylometry Analysis

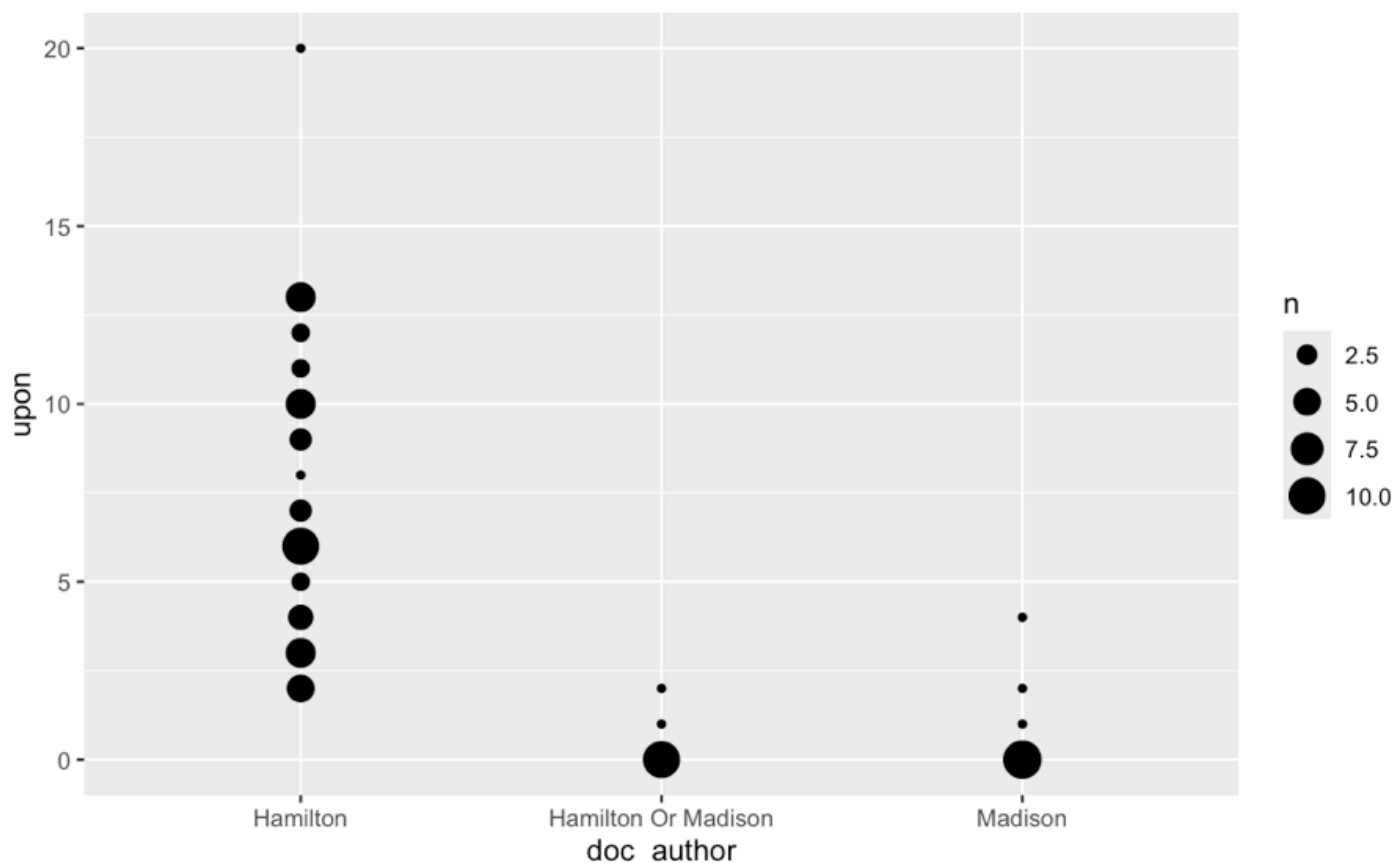
This workflow uses the frequency of function words to distinguish between two authors (Hamilton and Madison), which is the core principle of the Federalist Papers analysis.

This Federalist Papers example is satisfying in that it suggests that seeking data to find persuasive arguments is a valuable method for coming to conclusions. Sometimes it is not clear what data are pertinent. In the case of The Federalist Papers dispute, it would not be immediately obvious that counting the frequency of trivial words would be the road to decision on the authorship issue.

The evidence for the determination that Madison wrote the anonymous articles would be evaluated differently by Bayesian statisticians versus frequentist statisticians. A Bayesian would be willing to say that, given the word usage in The Federalist Papers, there is a 99.9% chance that it is written by Madison. Frequentists would say that the articles are either written by Madison or not. Both camps would probably agree to a statement something like this: “The probability is only 2.4% of getting no upons when randomly selecting a collection of 1000 words from a person’s writing that generally has 6 upons per 1000.”

Hide

```
ggplot(federalist, aes( x = doc_author, y = upon )) +
  geom_count()
```



Hide

```
word_count_cols <- names(federalist)[3:ncol(federalist)]

federalist_processed <- federalist %>%
  mutate(
    # Sum the counts across all word columns for each paper
    total_words = rowSums(select(., all_of(word_count_cols)))
  ) |>
  select(doc_author, upon, total_words) |>
  rename(Author = doc_author) |>
  mutate(upon_rate = (upon / total_words) * 1000)

federalist_processed
```

Author

upon

total_words

upon_rate

<chr>		<dbl>	<dbl>					<dbl>			
28	Hamilton	3	1094					2.7422303			
29	Hamilton	10	1348					7.4183976			
30	Hamilton	13	1298					10.0154083			
31	Hamilton	13	1128					11.5248227			
32	Hamilton	2	1026					1.9493177			
33	Hamilton	9	1143					7.8740157			
34	Hamilton	10	1470					6.8027211			
35	Hamilton	9	1526					5.8977720			
36	Hamilton	6	1826					3.2858708			
37	Madison	1	1798					0.5561735			
21-30 of 77 rows		Previous	1	2	3	4	5	6	...	8	Next

Hide

NA

Hide

```
# 1. Filter for known authors (Hamilton and Madison)
known_papers <- federalist_processed |>
  filter(Author %in% c("Hamilton", "Madison"))
# Rename for cleaner formula output
# 2. Perform the T-test
# H0: Mean usage rate of 'upon' is the same for both authors.
t_test_result <- t.test(upon_rate ~ Author, data = known_papers)

t_test_result
```

Welch Two Sample t-test

```
data: upon_rate by Author
t = 14.395, df = 62.935, p-value < 2.2e-16
alternative hypothesis: true difference in means between group Hamilton and group M
adison is not equal to 0
95 percent confidence interval:
 4.090045 5.408706
sample estimates:
mean in group Hamilton mean in group Madison
      4.9924219          0.2430467
```

The resulting p-value will provide the Frequentist conclusion on whether the word “upon” is a reliable fingerprint for attributing the disputed Federalist Papers.

Using data and statistical analyses will become an even more prominent part of our world in the future than it is now. The principal reason is the continuing development of computer technology. With the computer, it is now possible to deal with large databases and use techniques that would have been computationally impossible previously. Some such techniques involve simulation as a means to understand a collection of data. Often, such methods are computationally intensive and, consequently, become increasingly valuable as computer power increases.

Monte Carlo method

One such technique is called the Monte Carlo method, which involves using random processes by a computer to generate thousands of scenarios, enabling statistical techniques to derive the distribution of the behavior of a system. Today on a home computer, it is possible to do amazing statistical analyses essentially instantly. Early textbooks on statistics would emphasize methods for reducing computation. Now, with computers, those techniques are not so important.

[Hide](#)

```
known_papers_bayesian <- federalist_processed |>
  filter(Author %in% c("Hamilton", "Madison")) |>
  filter(!is.nan(upon_rate) & upon_rate > 0)
```

[Hide](#)

```
bayesian_model_upon <- stan_glm(
  upon_rate ~ Author,
  data = known_papers_bayesian,
  family = gaussian(),
  prior = normal(0, 5), # A weakly informative prior (standard for stylometry)
  chains = 2, iter = 1000
)
```

SAMPLING FOR MODEL 'continuous' NOW (CHAIN 1).

Chain 1:

Chain 1: Gradient evaluation took 0.000251 seconds

Chain 1: 1000 transitions using 10 leapfrog steps per transition would take 2.51 seconds.

Chain 1: Adjust your expectations accordingly!

Chain 1:

Chain 1:

Chain 1: Iteration: 1 / 1000 [0%] (Warmup)

Chain 1: Iteration: 100 / 1000 [10%] (Warmup)

Chain 1: Iteration: 200 / 1000 [20%] (Warmup)

Chain 1: Iteration: 300 / 1000 [30%] (Warmup)

Chain 1: Iteration: 400 / 1000 [40%] (Warmup)

Chain 1: Iteration: 500 / 1000 [50%] (Warmup)

Chain 1: Iteration: 501 / 1000 [50%] (Sampling)

Chain 1: Iteration: 600 / 1000 [60%] (Sampling)

Chain 1: Iteration: 700 / 1000 [70%] (Sampling)

Chain 1: Iteration: 800 / 1000 [80%] (Sampling)

Chain 1: Iteration: 900 / 1000 [90%] (Sampling)

Chain 1: Iteration: 1000 / 1000 [100%] (Sampling)

Chain 1:

Chain 1: Elapsed Time: 0.009 seconds (Warm-up)

Chain 1: 0.011 seconds (Sampling)

Chain 1: 0.02 seconds (Total)

Chain 1:

SAMPLING FOR MODEL 'continuous' NOW (CHAIN 2).

Chain 2:

Chain 2: Gradient evaluation took 6e-06 seconds

Chain 2: 1000 transitions using 10 leapfrog steps per transition would take 0.06 seconds.

Chain 2: Adjust your expectations accordingly!

Chain 2:

Chain 2:

Chain 2: Iteration: 1 / 1000 [0%] (Warmup)

Chain 2: Iteration: 100 / 1000 [10%] (Warmup)

Chain 2: Iteration: 200 / 1000 [20%] (Warmup)

Chain 2: Iteration: 300 / 1000 [30%] (Warmup)

Chain 2: Iteration: 400 / 1000 [40%] (Warmup)

Chain 2: Iteration: 500 / 1000 [50%] (Warmup)

Chain 2: Iteration: 501 / 1000 [50%] (Sampling)

Chain 2: Iteration: 600 / 1000 [60%] (Sampling)

Chain 2: Iteration: 700 / 1000 [70%] (Sampling)

Chain 2: Iteration: 800 / 1000 [80%] (Sampling)

Chain 2: Iteration: 900 / 1000 [90%] (Sampling)

Chain 2: Iteration: 1000 / 1000 [100%] (Sampling)

Chain 2:

Chain 2: Elapsed Time: 0.012 seconds (Warm-up)

Chain 2: 0.011 seconds (Sampling)

Chain 2: 0.023 seconds (Total)

Chain 2:

```
print(summary(bayesian_model_upon, digits = 3), pars = "beta")
```

Model Info:

```
function:      stan_glm
family:        gaussian [identity]
formula:       upon_rate ~ Author
algorithm:     sampling
sample:        1000 (posterior sample size)
priors:        see help('prior_summary')
observations:  54
predictors:    2
```

Estimates:

	mean	sd	10%	50%	90%
(Intercept)	4.969	0.299	4.603	4.962	5.350
AuthorMadison	-3.655	1.175	-5.172	-3.657	-2.151
sigma	2.106	0.221	1.846	2.086	2.404

Fit Diagnostics:

	mean	sd	10%	50%	90%
mean_PPD	4.782	0.409	4.283	4.767	5.307

The mean_ppd is the sample average posterior predictive distribution of the outcome variable (for details see `help('summary.stanreg')`).

MCMC diagnostics

	mcse	Rhat	n_eff
(Intercept)	0.011	0.999	767
AuthorMadison	0.036	0.999	1089
sigma	0.008	0.998	773
mean_PPD	0.015	1.001	732
log-posterior	0.082	1.012	229

For each parameter, mcse is Monte Carlo standard error, n_eff is a crude measure of effective sample size, and Rhat is the potential scale reduction factor on split chains (at convergence Rhat=1).

```
posterior_samples <- as_tibble(bayesian_model_upon)
```

```
mean_diff <- posterior_samples %>%
  summarise(
    MeanDifference = mean(AuthorMadison),
    ProbMadisonHigher = mean(AuthorMadison > 0)
  )

mean_diff
```

MeanDifference <dbl>	ProbMadisonHigher <dbl>
-3.655264	0
1 row	

The output of the MCMC process (posterior_samples) is the simulated distribution of the parameter values.

- The MeanDifference tells you the posterior estimate of how much higher Madison’s usage rate of “upon” is compared to Hamilton’s.
- The ProbMadisonHigher (which will likely be close to 1.0) is the ultimate Bayesian conclusion: it’s the probability that the rate of “upon” usage for Madison is truly higher than Hamilton’s.

The strength of the Bayesian Monte Carlo method is that it provides a direct probability statement ($P(\text{Madison} \mid \text{Data})$), rather than just a p-value ($P(\text{Data} \mid H_0)$).

Schools of thought with common basics

We have seen statistics as having two basic parts: - Organizing, describing, and summarizing a collection of data when we know all the data.

- Inferring information about the whole population when we have data about only a sample of the population. When we make an estimate of the value of a feature of the whole population given data about a sample, our challenge is to describe how accurate our estimate is likely to be by answering the following questions:
- How close is our estimate to the correct value?
- How confident are we that our estimate is, in fact, that close?

Hide

```
tibble(Approach = c("Frequentist (Classical)", "Bayesian"),
       Key_idea = c("The p-value. Probability is the long-run frequency. Parameters
are fixed but unknown. Inference relies on data alone.", "Probability is degree of
belief. Parameters are random variables. Inference updates prior beliefs with new d
ata to form a posterior distribution."),
       R_Focus = c("t.test(), lm(), chisq.test().", "rstanarm or brms packages"))
```

Approach

<chr>

Frequentist (Classical)

Bayesian

2 rows | 1-1 of 3 columns

Statistics is a powerful tool for understanding our world. We end the course with a statistic about the course: Among people who learn something about statistics, 100% appreciate our world with more clarity.

Concept and Applications

STATISTICS YOUR WAY WITH CUSTOM FUNCTIONS

One of the coolest things you can do in R is write your own functions. Custom functions allow you to define a specific action you are interested in, which you can then easily apply to new data. This can be anything from calculating a unique statistic, to creating a custom plot, to combining several outputs into a single display.

WRITING CUSTOM FUNCTIONS

To make a custom function, you need to define one or more inputs followed by some action and an output. Your function will take the input and perform the action based on that input and spit out some output in the form of transformed data, summary statistics, or a plot.

For example, let's create a custom function that replicates the "mean()" function in R:

Hide

```
mean.fun <- function(x)
{
  my.mean <- sum(x) / length(x)
  return(my.mean)
}
```

Once you've defined a function and given it a name, you can then use the function on new data:

Hide

```
mean.fun(0:20)
```

```
[1] 10
```

Hide

```
mean.fun(rnorm(300))
```

```
[1] -0.01826186
```

While the function "mean.fun" only had 1 input (the vector x), we can also define multiple inputs. For example, let's adjust the function so that it has an input "delete.outliers," which, when true, automatically deletes outliers before calculating the mean. This will come in handy whenever you do exploratory data analysis.

Hide


```
mean.fun <- function(x, delete.outliers = F)
{
  if(delete.outliers == F) {
    my.mean <- sum(x) / length(x)
  }
  if(delete.outliers == T) {
    Orig.Mean <- sum(x) / length(x)
    Orig.SD <- sd(x)
    Outlier.log <- (x > Orig.Mean + 2 * Orig.SD) | (x <
    Orig.Mean - 2 * Orig.SD)
    x.noout <- x[Outlier.log == FALSE]
    my.mean <- sum(x.noout) / length(x.noout)
  }
  return(my.mean)
}
```

Here is an example of the function in action. Let's create a vector "data" with 100 samples from a standard normal distribution with 1 outlier at 400.

Hide

```
Data <- c(rnorm(n = 100, mean = 0, sd = 1),400)
mean.fun(Data, delete.outliers = F)
```

```
[1] 3.978438
```

Hide

```
mean.fun(Data, delete.outliers = T)
```

```
[1] 0.01822285
```

Once we've defined a custom function in R, we can then use that function in other functions.

MODIFYING EXISTING FUNCTIONS

If you find that you're frequently adding certain things to plots, such as reference lines for the median or mean, then creating a custom plotting function can make things easier for you.

Hide

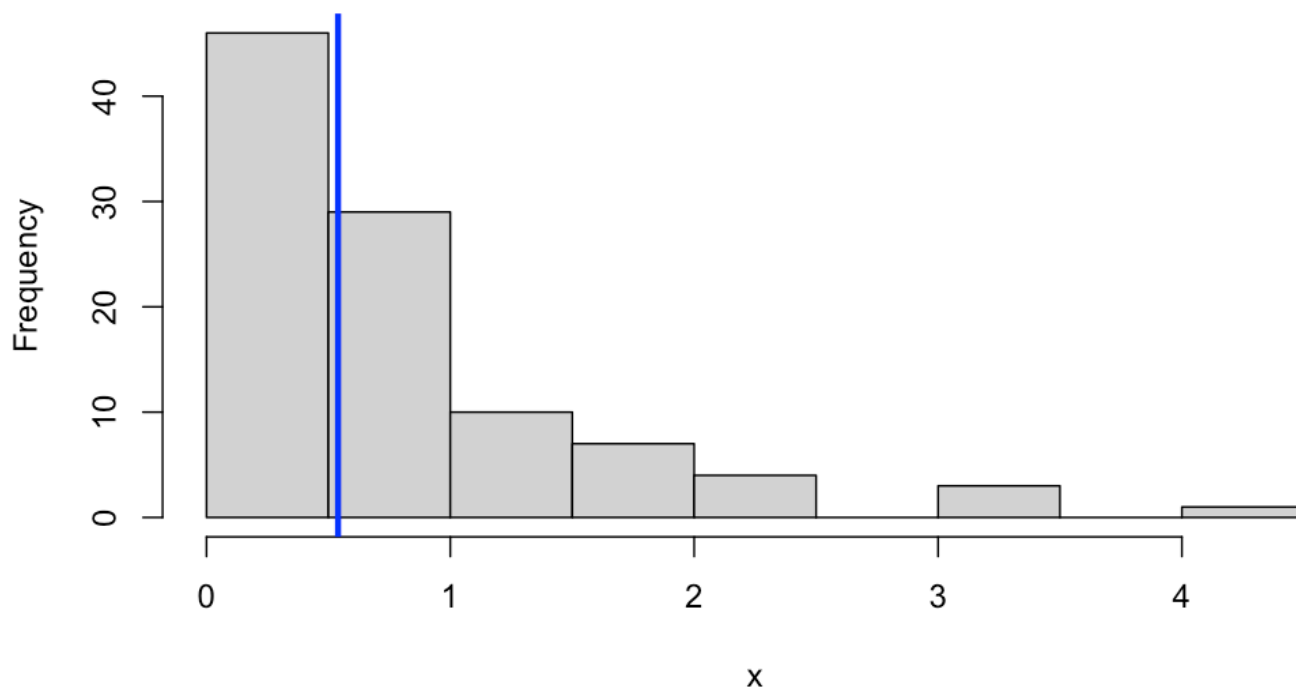
```
hist.fun <- function(x, add.median = T) {
  hist(x)
  if(add.median == T) {abline(v = median(x), lwd = 3, col
= "blue")}
}
```

Use "abline" to define any line, and then "v =" for a vertical line.

Hide

```
hist.fun(rexp(100), add.median = T)
```

Histogram of x

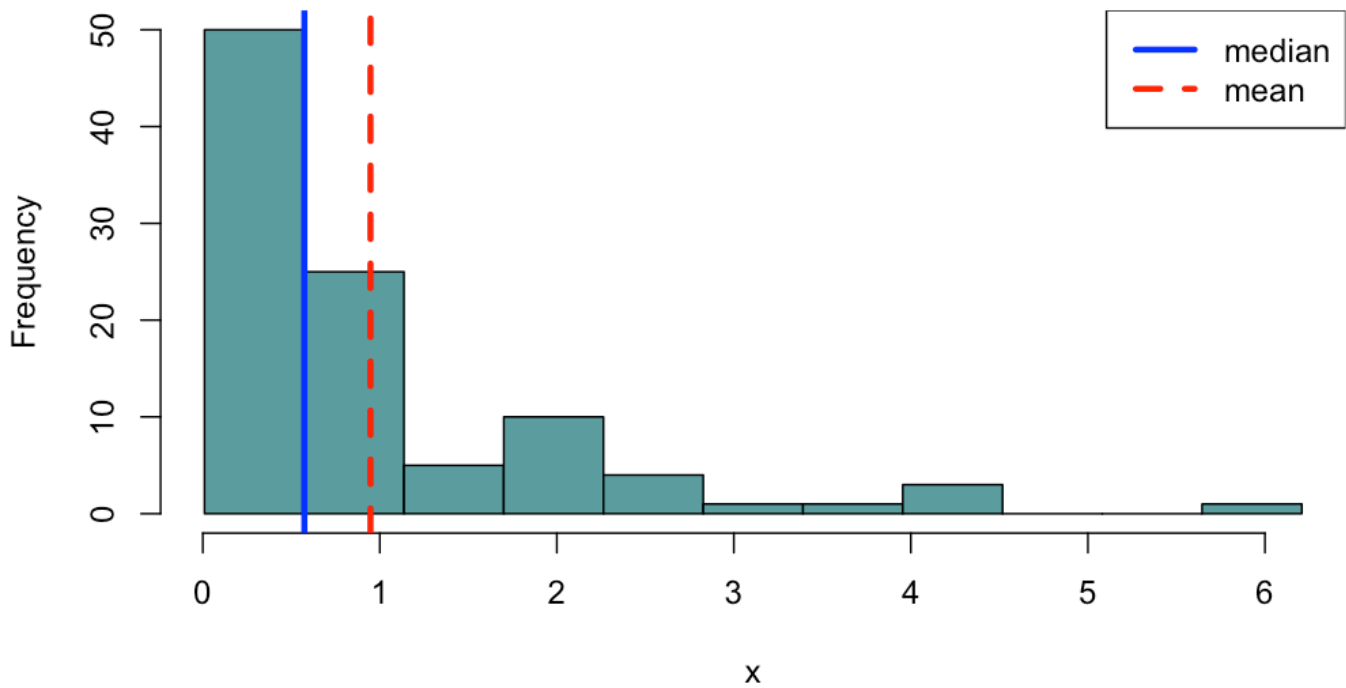


Let's modify our custom histogram function "hist.fun":

Hide

```
hist.fun <- function(x, add.median = T, add.mean = T,
  add.legend = T, num.breaks = 12) {
  b <- seq(min(x), max(x), length=num.breaks)
  hist(x, col="cadetblue", breaks = b)
  if(add.median == T) {abline(v = median(x), lwd = 3, col
    = "blue")}
  if(add.mean == T) {abline(v = mean(x), lty = 2, lwd = 3,
    col = "red")}
  if(add.legend == T) {
    legend("topright", c("median", "mean"), lwd = c(3, 3),
    lty = c(1, 2), col = c("blue","red"))
  }
}
hist.fun(rexp(100), add.median = T,
  add.mean = T, add.legend = T)
```

Histogram of x



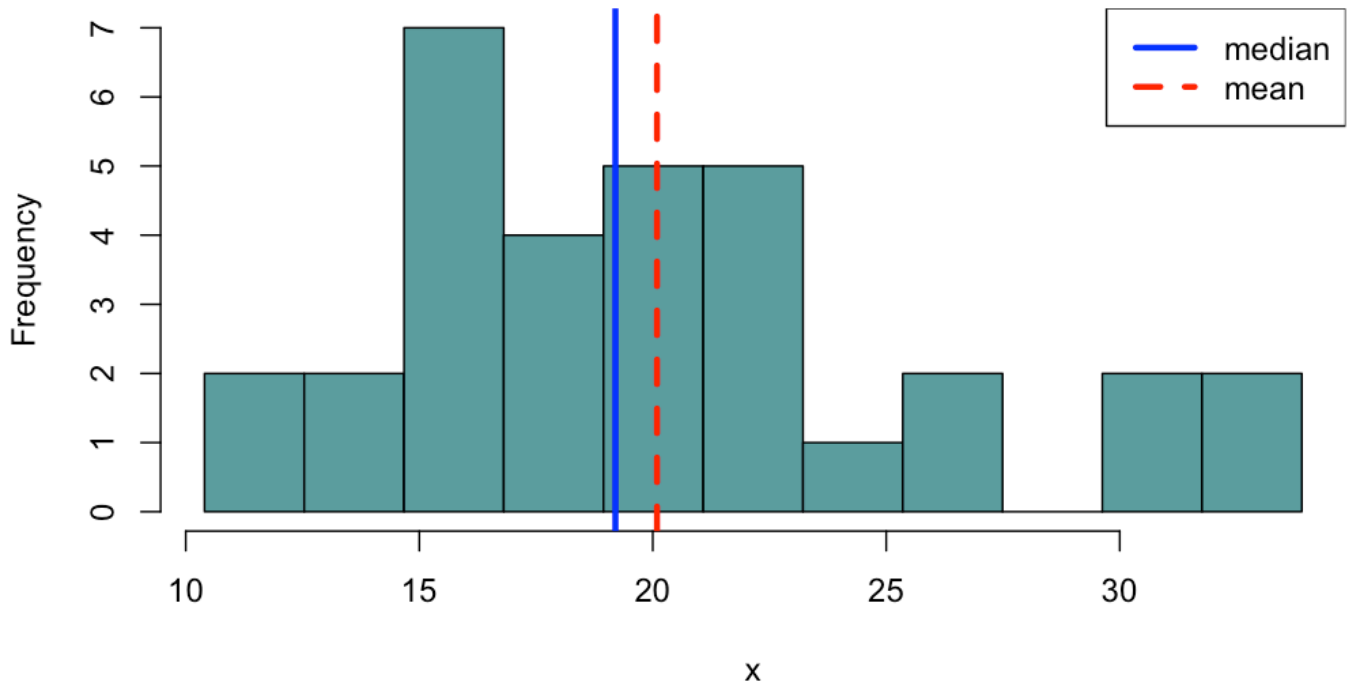
Let's try this function on another dataset.

Hide

```
hist.fun <- function(x, add.median = T, add.mean = T,
  add.legend = T, num.breaks = 12) {
  b <- seq(min(x), max(x), length=num.breaks)
  hist(x, col="cadetblue", breaks = b)
  if(add.median == T) {abline(v = median(x), lwd = 3, col
    = "blue")}
  if(add.mean == T) {abline(v = mean(x), lty = 2, lwd = 3,
    col = "red")}
  if(add.legend == T) {
    legend("topright", c("median", "mean"), lwd = c(3, 3),
    lty = c(1, 2), col = c("blue","red"))
  }
}

hist.fun(mtcars$mpg, add.median = T, add.mean = T, add.legend = T)
```

Histogram of x



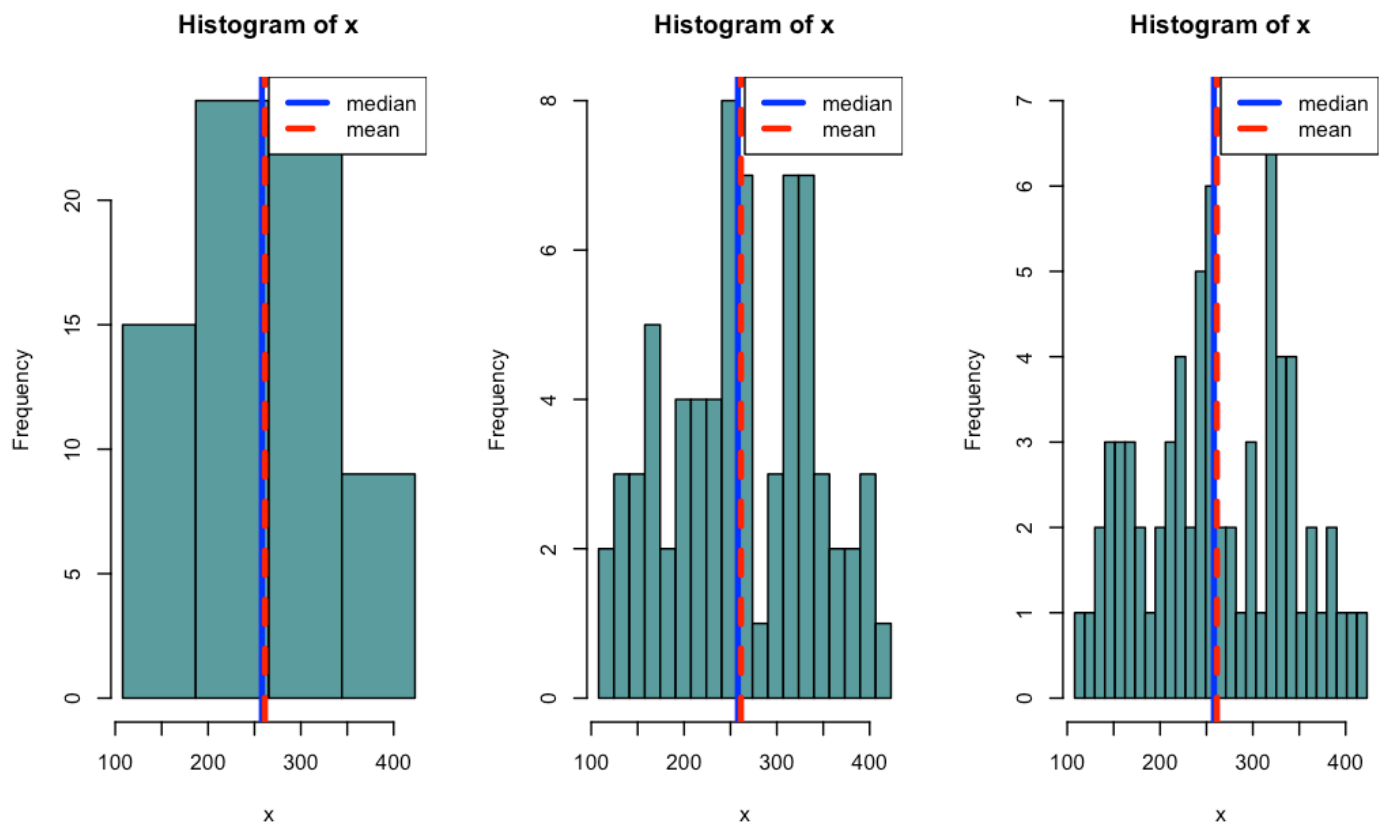
We can generate plots with different breaks so that we can compare them.

Hide

```
par(mfrow=c(1,3))
hist.fun(chickwts$weight,num.breaks = 5)
hist.fun(chickwts$weight,num.breaks = 20)
```

Hide

```
hist.fun(chickwts$weight,num.breaks = 30)
```



We can add as many custom procedures and plotting parameters as we'd like. In fact, we could create a similar function for scatterplots.

We can create a custom scatterplot with some fun features.

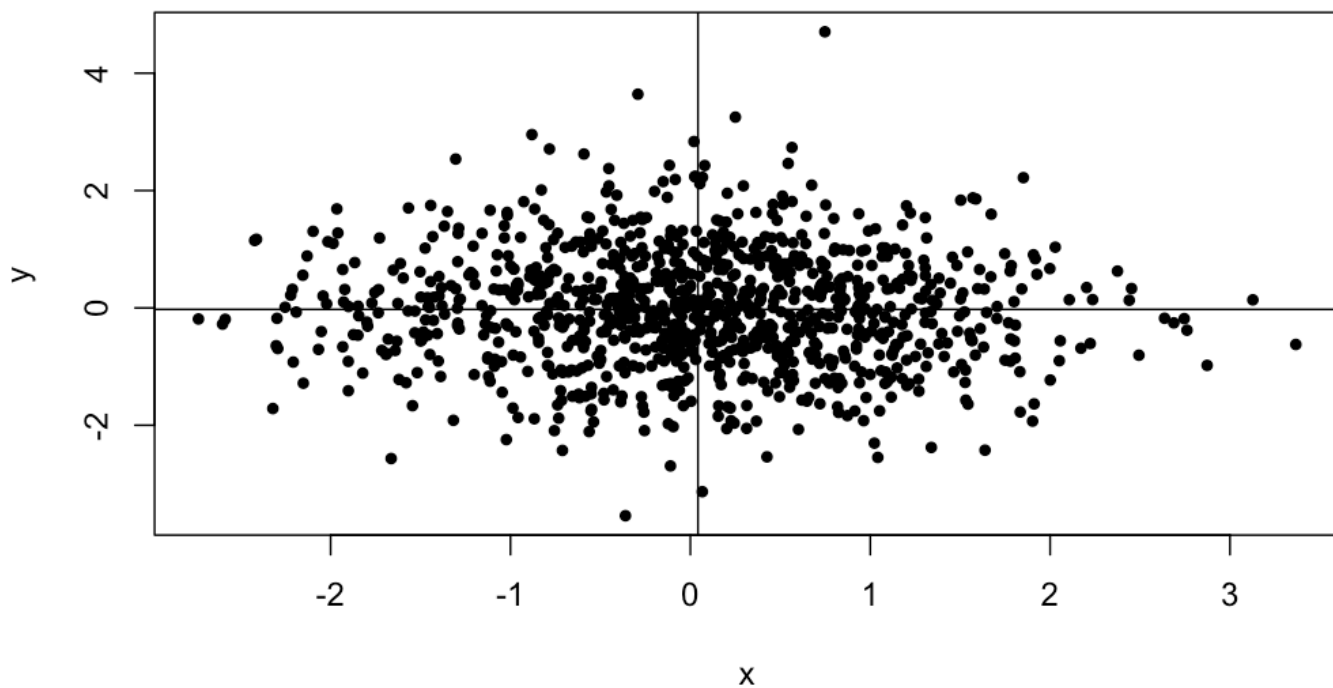
Hide

```
scatter.fun <- function(x = rnorm(1000), y = rnorm(1000),
  add.mean = T) {
  plot(x, y, pch=20)
  if(add.mean == T) {
    abline(h = mean(y))
    abline(v = mean(x))
  }
}
```

Let's run it with defaults.

Hide

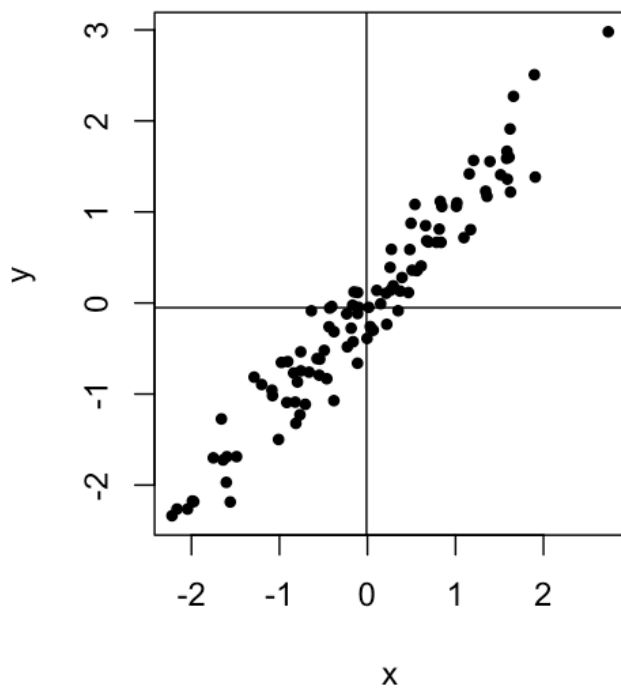
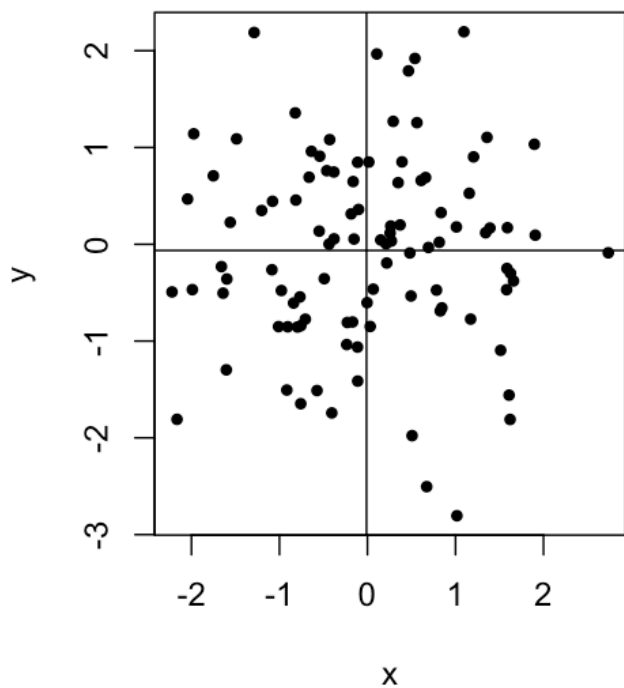
```
scatter.fun()
```



Let's see our custom scatterplot in action. We can create 2 plots, one where there is not significant correlation and one where there is.

Hide

```
x <- rnorm(100)
y.uncorr <- rnorm(100)
y.corr <- x + rnorm(100, 0, .3)
par(mfrow = c(1, 2))
scatter.fun(x, y.uncorr)
scatter.fun(x, y.corr)
```



“Add.regression” adds a regression line. If the correlation is significant, we’ll make the line green; if not, it will be red.

Hide

```

scatter.fun <- function(x = rnorm(1000), y = rnorm(1000),
  add.mean = T, add.regression = T,
  add.conclusion = T) {
  plot(x, y, pch=20)
  if(add.mean == T) {
    abline(h = mean(y))
    abline(v = mean(x))
  }
  if(add.conclusion == T) {
    C.Test <- cor.test(x, y)
    Coefficient <- C.Test$estimate
    p.value <- C.Test$p.value
    if(p.value <= .05) {
      Conclusion <- paste("The correlation coefficient is ",
        round(Coefficient, 2),
        ".\nIt IS significantly different from 0 (p = ",
        round(p.value, 3),
        ")", sep = "")
    }
    if(p.value > .05) {
      Conclusion <- paste("The correlation coefficient is ",
        round(Coefficient, 2),
        ".\nIt is NOT significantly different from 0
        (p = ", round(p.value, 3),
        ")", sep = "")
    }
    mtext(text = Conclusion,
      side = 3, line = 1, cex = .8) }

  if(add.regression == T) {
    Model <- lm(y ~ x)
    p.value <- anova(Model)$"Pr(>F)"[1]
    if(p.value <= 0.05) {Reg.Line.Col <- "green"}
    if(p.value > 0.05) {Reg.Line.Col <- "darkred"}
    abline(lm(y ~ x), lty = 2, lwd = 4, col = Reg.Line.Col)
  }
}

```

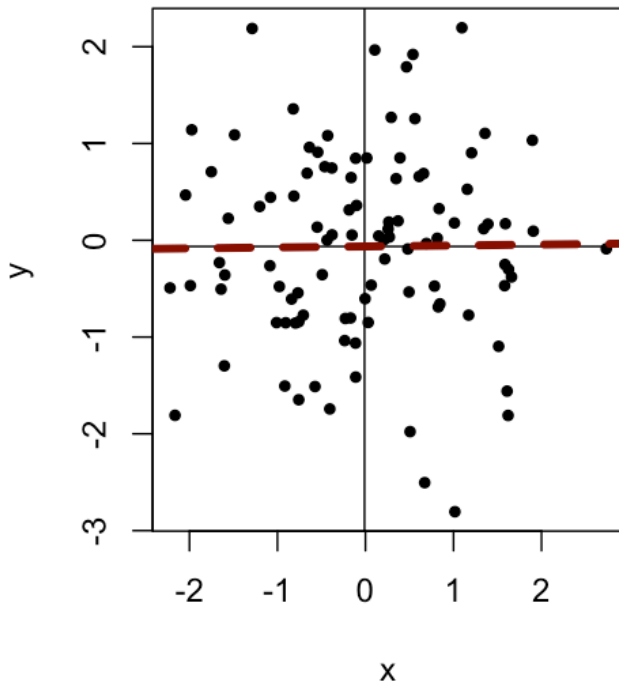
Hide

```

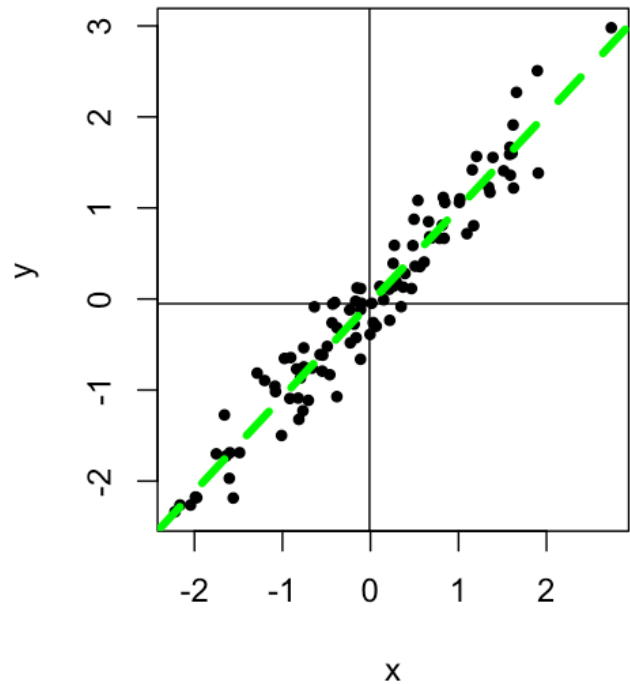
par(mfrow = c(1, 2))
scatter.fun(x, y.uncorr)
scatter.fun(x, y.corr)

```


The correlation coefficient is 0.01.
It is NOT significantly different from 0
($p = 0.916$)



The correlation coefficient is 0.97.
It IS significantly different from 0 ($p = 0$)



PITFALL

- R has built-in functions, such as “summary()” or “median()”, and this overlap is the most common pitfall: overwriting a built-in R function.
- Some of R’s built-in functions are common words, such as “contour,” “image,” “jitter,” “length,” “mean,” “median,” “order,” “plot,” “sample,” “sign,” “sequence,” “sort,” and “summary.”

Before creating a new function, always type “?(new function name)” to see if it already exists as an R function.